Epistemic Handoffs in Multi-Agent Software Engineering: Empirical Evaluation of Mid-Task Frontier Takeover on SWE-bench Verified

Authors: Keisho Research Working Group (Lead: bagnesium) [Review: attribution unconfirmed — confirm actual authorship before any external sharing]
Status: COMPLETED PILOT MANUSCRIPT (EMPIRICAL FIVE-WAY FRONTIER TAKEOVER COMPARISON)
Date: October 27, 2026
Evaluation Target: SWE-bench Verified (`pydata__xarray-6721`)

ABSTRACT

We report a completed empirical five-way comparison evaluating agent context handoff representations during a mid-task takeover between two distinct frontier model families on a complex software repair task from SWE-bench Verified (`pydata__xarray-6721`). The study evaluates five context transfer conditions across an identical checkpoint: `REPOSITORY_ONLY` (no handoff context; testbed exploration from scratch), `FULL_HISTORY` (uncompressed turn-by-turn conversation), `SUMMARY_2K` (2,048-token LLM narrative summary), `STRUCTURED_NOTES_2K` (7-field schema notes + deterministic facts), and `KEISHO_2K` (compiler-generated typed items with file-revalidation scoping).

Unlike earlier pilot studies where the predecessor reached a solved checkpoint (measuring solution preservation), the present study implements a *pre-execution source-edit interception protocol*: the predecessor (`openai/gpt-6-sol`) explores the repository and formulates a repair strategy, but is paused strictly prior to executing its first attempted source code modification. This leaves the container working tree unpatched. At runtime, the SWE-bench evaluation harness classified the empty checkpoint patch via its native empty-patch shortcut (recorded as `status: unresolved` , `empty_patch: true`). To close the empirical verification gap, dedicated container testing executed the official `FAIL_TO_PASS` and `PASS_TO_PASS` pytest suites directly on the unpatched codebase inside Docker, confirming that the target defect failed with `UnexpectedDataAccess: Tried accessing data` at `xarray/core/common.py:2026` while all 1,405 baseline regression tests passed.

Following checkpoint verification, context was transferred to an asymmetric frontier successor (`anthropic/claude-opus-5.5`) across all five branches. All five successor conditions successfully resolved the task (5/5 `resolved` by official Docker SWE-bench evaluation). Because all five conditions resolved the issue, this experiment does not demonstrate differential task accuracy. However, substantial operational differences were observed across representations:

1. **Token Efficiency**: `FULL_HISTORY` consumed 138,443 successor input tokens across 4 calls. In contrast, `SUMMARY_2K` consumed 44,126 input tokens (68.1% reduction vs. `FULL_HISTORY`), `STRUCTURED_NOTES_2K` consumed 45,429 input tokens (67.2% reduction), `KEISHO_2K` consumed 47,170 input tokens (65.9% reduction), and `REPOSITORY_ONLY` consumed 54,526 input tokens (60.6% reduction).
2. **Turn Efficiency**: `KEISHO_2K` and `FULL_HISTORY` required the joint-fewest model calls (4 model calls, 3 tool calls), moving directly from the handoff to source modification. `SUMMARY_2K` and

`STRUCTURED_NOTES_2K` each required 5 model calls (4 tool calls), taking an additional exploratory verification step before editing. `REPOSITORY_ONLY` required 7 model calls (6 tool calls), spending 3 calls re-investigating the repository from scratch.

3. **Execution Latency**: Wall-clock duration was 32.7 seconds for `FULL_HISTORY` and 34.5 seconds for `KEISHO_2K`, compared to 53.8 seconds for `REPOSITORY_ONLY`, 86.5 seconds for `STRUCTURED_NOTES_2K`, and 86.8 seconds for `SUMMARY_2K`. Wall-clock time reflects composite provider response latency and turn count, rather than pure context processing speed.

Actual provider billing for the completed comparison run was **$1.534741** (conservative accounted spend: **$1.590336**; unreconciled reservations: **$0.00**). Billed spend across all five iterative experimental attempts was **$2.051311**, plus one unreconciled reservation of **$0.379008** from an upstream routing parameter rejection. As an $N=1$ single-task study, these results do not establish general performance superiority, but provide an empirical demonstration of how structured epistemic handoffs perform during mid-task frontier agent transfer.

---

1. Introduction and Research Motivation

Autonomous software engineering agents increasingly require multi-agent collaboration, delegation, and context handoff. When tasks exceed a single model's context window, or when specialized models are paired—for instance, using an analytical model for codebase triage and a code-specialized frontier model for implementation—intermediate state must be transferred accurately.

Prior work has highlighted *Handoff Debt* (KC & Budathoki, 2026), where informal or unstructured summaries omit crucial negative evidence, obscure provenance, and induce hallucinations or redundant exploration in successor agents. In an earlier single-task pilot on `sympy__sympy-20590`, all five successor conditions preserved task resolution. However, because the predecessor's cumulative patch had already resolved the task prior to handoff, that initial study evaluated *solution preservation* rather than the *continuation of unfinished work*.

To investigate context handoffs during active, unfinished problem-solving, this study establishes an empirical protocol evaluating a genuine mid-task takeover between two distinct frontier model families on a complex repair instance:

1. **Multi-File Benchmark Target**: Moving from a single-file SymPy issue to `pydata__xarray-6721`, a multi-file defect in `xarray/core/common.py` and `xarray/core/variable.py` from SWE-bench Verified.
2. **Frontier Model Asymmetry**: Pairing OpenAI's `gpt-6-sol` as predecessor investigator with Anthropic's `claude-opus-5.5` as successor repair engineer, mediated by `gpt-6-luna` for narrative and schema-based note generation.
3. **Pre-Execution Interception Stopping Rule**: Enforcing an automated interception barrier on the predecessor's first attempted non-test source edit, halting the agent prior to tool execution and leaving the working tree completely unpatched.
4. **Empirical Checkpoint Verification**: Confirming through direct container execution that the unpatched baseline reproduces the target defect while preserving the wider test suite before initiating successor branches.

---

2. Experimental Design and Interception Protocol

The experimental architecture evaluates five context transfer conditions across an identical checkpoint. All runs were conducted in isolated Linux containers under explicit cost bounds.

2.1 Benchmark Target: `pydata__xarray-6721`

- **Instance ID**: `pydata__xarray-6721` (SWE-bench Verified)
- **Repository**: `pydata/xarray`
- **Base Commit**: `cc183652bf6e1273e985e1c4b3cba79c896c1193`
- **Defect Description**: Accessing `Dataset.chunks` or `Dataset.chunksizes` on lazily indexed or inaccessible storage backends unintentionally forces array materialization into memory. The underlying flaw resides in `get_chunksizes()` in `xarray/core/common.py` querying `hasattr(v.data, "chunks")` rather than checking `v.chunks is not None`, alongside a corresponding access in `Variable.chunksizes` (`xarray/core/variable.py`).
- **Official Test Suite**:
- `FAIL_TO_PASS`: `xarray/tests/test_dataset.py::TestDataset::test_chunks_does_not_load_data` (1 test).
- `PASS_TO_PASS`: 1,405 regression tests in `xarray/tests/test_dataset.py`.

2.2 Frontier Multi-Model Pipeline

The experiment uses three distinct models across task phases:

- **Predecessor Investigator**: `openai/gpt-6-sol` (via OpenRouter, reasoning effort `none`). Configured rates: prompt $2.00/M, completion $10.00/M, cache-write reservation $2.50/M.
- **Handoff Note Generators**: `openai/gpt-6-luna` (via OpenRouter, reasoning effort `none`). Configured rates: prompt $0.10/M, completion $0.50/M, cache-write reservation $0.125/M.
- **Successor Repair Engineers (5 conditions)**: `anthropic/claude-opus-5.5` (via OpenRouter, reasoning effort `low` required by provider catalog). Configured rates: prompt $4.00/M, completion $20.00/M, cache-write reservation $5.00/M.

2.3 Pre-Execution Interception Stopping Rule

Heuristic stopping rules based on string-matching error logs are prone to failure because language models often trap expected errors inside `try...except` blocks or shell scripts that exit with code 0. To provide a robust stopping boundary, the system intercepts the predecessor's actions:

1. The predecessor agent is initialized with the problem statement and full read/command access to the testbed repository.
2. The agent inspects files, runs shell searches, and executes reproductions.
3. When the predecessor issues an action targeting a non-test source-code modification (e.g., editing `xarray/core/common.py`), the execution harness intercepts the action *before* the modification is written to disk.
4. The tool returns an interception notice (`status: paused_before_execution`).
5. The predecessor session halts cleanly with `reason: checkpoint`.
6. The container working tree is verified clean (`git diff` is empty).

7. The unpatched checkpoint state is submitted to grading to confirm it is unresolved before any successor branch commences.

2.4 Five Successor Context Conditions

All five successor branches execute from the clean checkpoint state, receiving identical tool sets (`benchmark-agent/5.0` with `json_object/3.0` action wire), identical task descriptions, and identical container environments:

1. **`REPOSITORY_ONLY`**: Receives no predecessor context (prompt contains only: *"No handoff context is provided for this run. The repository is available at /testbed; begin work on the issue above using the tools."*).
2. **`FULL_HISTORY`**: Receives the raw, uncompressed sequence of all predecessor conversation turns, tool calls, and outputs (16,166 tokens; 55,749 characters).
3. **`SUMMARY_2K`**: Receives a 2,048-token capped free-form narrative summary produced by GPT-6 Luna (326 tokens; 1,437 characters).
4. **`STRUCTURED_NOTES_2K`**: Receives a 7-field structured schema (problem understanding, work completed, evidence observed, observed failures, remaining uncertainties, rollback notes, recommended next actions) plus deterministic repository continuation facts produced by GPT-6 Luna (435 tokens; 2,137 characters).
5. **`KEISHO_2K`**: Receives compiler-verified, typed epistemic items (observed facts, hypothesized claims, unknown boundaries, provenance pointers) scoped to the repository state (2,035 tokens; 8,103 characters).

---

3. Engineering Discoveries and Iterative Development

Conducting mid-task handoffs across heterogeneous frontier model families required addressing several real-world distributed agent challenges across five successive attempts:

3.1 Disclosure of All Experimental Attempts

Five attempts were conducted under the authorized spending cap:

- **Attempt 1**: Tested a post-reproduction error detection heuristic. The predecessor successfully reproduced the defect using a Python snippet, but caught the exception inside a `try...except` block. Because the shell exited with return code 0 and non-matching text, the detector missed the checkpoint, and the predecessor proceeded past the desired stopping point. Spend: $0.069326.
- **Attempt 2**: Refined the stdout regex detector. The model adapted its output format, again bypassing heuristic detection before attempting an edit. Spend: $0.065966.
- **Attempt 3**: Replaced heuristic log parsing with the pre-execution source-edit interception protocol. The predecessor halted cleanly at the first source edit. However, the first successor call to Claude Opus 5.5 failed immediately with an HTTP 404 from OpenRouter (`Filter by Parameters`) because the request passed an explicit `seed: 20261007` with `require_parameters: True`, which Anthropic's provider endpoint does not support. The spend guard held a $0.379008 reservation for the rejected request. Spend: $0.036516 billed (plus $0.379008 unreconciled reservation).
- **Attempt 4**: Omitted the `seed` parameter for Anthropic models. The predecessor halted cleanly, but the first successor call was halted by local client-side token reservation guards: Claude Opus 5.5's native tokenizer returned 9,540 input tokens on a request that OpenAI's `o200k_base` tokenizer measured at 5,990 tokens.

Because the returned usage exceeded the pre-declared reservation upper bound, the local harness failed closed. Spend: $0.344762.

- **Attempt 5 (Completed Run)**: Scaled the Anthropic token reservation multiplier to 2.0x (`int(math.ceil(base * 2.0))`) and implemented explicit empty-patch handling for checkpoint evaluation. All 33 model calls completed end-to-end across the predecessor, note generators, and all five successor branches. Spend: $1.534741 billed.

3.2 Tokenizer Density Divergence Across Providers

OpenAI models use the `o200k_base` Byte-Pair Encoding (BPE) vocabulary (~200,000 tokens), whereas Anthropic's Claude family uses a distinct vocabulary (~65,000 tokens). On technical Python source code and SWE-bench prompts, Claude's tokenizer exhibited an empirical expansion ratio of **1.575x to 1.633x** relative to `o200k_base`. Multi-agent architectures that calculate token reservations using a single tokenizer will systematically under-reserve for denser tokenizers, causing client-side validation failures unless provider-specific scaling factors are enforced.

3.3 Pre-Execution Interception Boundary

The pre-execution edit interception mechanism eliminated heuristic uncertainty. By hooking the tool dispatcher before disk writes execute, the harness precisely separated the diagnostic/triage phase from the modification phase without depending on fragile log patterns.

3.4 Successor Seed Non-Determinism

Because Anthropic's API via OpenRouter does not accept the `seed` parameter, successor branches operated without server-side seed pinning. While temperature was set to 0.0 across all calls, upstream provider routing and distributed sampling mean that outputs remain subject to stochastic variance.

---

4. Empirical Evaluation Results

The completed benchmark comparison was executed in Attempt 5 under authorization ceiling $20.00.

4.1 Predecessor Trajectory and Checkpoint State

Predecessor `gpt-6-sol` executed 6 model calls (25.0s elapsed):

- **Calls 1–2**: Examined `xarray/core/dataset.py` and `xarray/core/common.py` using `grep` and `sed` (noting `rg` was not installed), tracing `Dataset.chunks` to `get_chunksizes()`.
- **Calls 3–4**: Inspected `xarray/core/variable.py` to check the behavior of `Variable.chunks` vs `Variable.data` and `chunksizes`.
- **Call 5**: Executed a local reproduction script using a custom `LazilyIndexedArray` whose `__getitem__` raised `RuntimeError("loaded")`. Accessing `ds.chunks` triggered `RuntimeError: loaded`, confirming that accessing chunk metadata coerced array loading.
- **Call 6**: Formulated a shell script to patch `xarray/core/common.py`. The action was intercepted before execution, returning `status: paused_before_execution`. Predecessor terminated with `reason: checkpoint`.

4.2 Checkpoint Verification

1. **Runtime Evaluator Classification**: During benchmark execution, the harness submitted the predecessor's empty patch to SWE-bench evaluation (`swebench eval`). The SWE-bench evaluation harness recognized the

empty patch natively, recording `empty_patch_instances: 1`, `empty_patch_ids: ["pydata__xarray-6721"]`, and `status: unresolved`. The harness did not spin up a Docker container for test execution at that stage.

2. **Empirical Container Test Execution**: To verify that the unpatched state genuinely failed the task's tests (rather than relying solely on the empty-patch labeling shortcut), a post-run verification command executed the official evaluation script directly inside the task's Docker image (`swebench/sweb.eval.x86_64.pydata_1776_xarray-6721@sha256:58c7f4ada7f3cbcfcde50e0b55157701100cf694faa87978c672e368c0531898`).

- The official test patch was applied to `xarray/tests/test_dataset.py`, adding `test_chunks_does_not_load_data`.
- Pytest executed across all 1,406 collected items in `xarray/tests/test_dataset.py`.
- **Target Defect Outcome (`FAIL_TO_PASS`)**: `test_chunks_does_not_load_data` FAILED with:

```
xarray.tests.UnexpectedDataAccess: Tried accessing data

/testbed/xarray/core/common.py:2026: in get_chunksizes: if hasattr(v.data, "chunks"):
```

- **Baseline Regression Outcome (`PASS_TO_PASS`)**: All 1,405 baseline tests passed (1,405 passed, 33 skipped, 1 xfailed, 5 xpassed in 29.18s).
- **Conclusion**: The unpatched codebase cleanly reproduced the genuine task failure with zero environment, dependency, or configuration errors, empirically confirming that the checkpoint was genuinely unresolved prior to successor takeover.

4.3 Five-Way Successor Takeover Results

All five successor branches were executed from the identical unpatched checkpoint and evaluated using the official SWE-bench evaluation harness in Docker:

Condition	Official Docker Status	Model Calls	Tool Calls	Successor Input Tokens	Successor Output Tokens	Wall-Clock Time (s)	Successor Modification Strategy
FULL_HISTORY	**resolved**	4	3	138,443	1,585	32.7s	Replaced `hasattr(v.data, 'chunks')` with `v.chunks is not None` in `common.py` and `variable.py`
SUMMARY_2K	**resolved**	5	4	44,126	1,931	86.8s	Replaced `v.data` with `v._data` in `common.py` and `variable.py`
REPOSITORY_ONLY	**resolved**	7	6	54,526	2,315	53.8s	Replaced `v.data` with `v._data` in `common.py`
STRUCTURED_NOTES_2K	**resolved**	5	4	45,429	1,650	86.5s	Replaced `v.data` with `v._data` in `common.py` and `variable.py`
KEISHO_2K	**resolved**	4	3	47,170	1,493	34.5s	Replaced `v.data` with `v._data` in `common.py` and `variable.py`

4.4 Handoff Payload Content and Information Disclosure

A critical qualitative factor is how each handoff representation handled the predecessor's intercepted patch proposal:

- **`FULL_HISTORY`**: Transmitted the raw text of the predecessor's Call 6 tool invocation, including the exact proposed diff and the subsequent system notice: `[PAUSED: source code edit intercepted prior to execution]`.
- **`SUMMARY_2K`**: Explicitly summarized the proposed patch: *"Proposed fix: change get_chunksizes() to use v.chunks and iterate zip(v.dims, v.chunks)... The predecessor attempted to apply those edits... but the edit was intercepted and paused before execution. No source files were changed."*
- **`STRUCTURED_NOTES_2K`**: Explicitly documented the proposed fix in `work_completed` while recording `repository_change_state: "no source changes observed"` in the deterministic facts.
- **`KEISHO_2K`**: Emitted verified tool outputs and typed hypotheses. The unexecuted edit tool call was excluded from tool observations because it never executed. The need to guard lazy arrays was preserved as an unverified hypothesis (`S12 [HYPOTHESIZED]`), while acceptance criteria and workspace changes were explicitly flagged as unknown.

- **`REPOSITORY_ONLY`**: Contained zero context. Successor Opus 5.5 re-discovered the root cause independently by searching `xarray/core/common.py`.

---

5. Quantitative and Qualitative Analysis

5.1 Context Token Consumption

Carrying uncompressed conversation turns in `FULL_HISTORY` resulted in extreme token accumulation, consuming **138,443 input tokens** across 4 calls:

- `FULL_HISTORY`: 138,443 input tokens (100.0% baseline)
- `REPOSITORY_ONLY`: 54,526 input tokens (60.6% reduction)
- `KEISHO_2K`: 47,170 input tokens (65.9% reduction)
- `STRUCTURED_NOTES_2K`: 45,429 input tokens (67.2% reduction)
- `SUMMARY_2K`: 44,126 input tokens (68.1% reduction)

The initial handoff token sizes were:

- `FULL_HISTORY`: 16,166 tokens
- `KEISHO_2K`: 2,035 tokens
- `STRUCTURED_NOTES_2K`: 435 tokens
- `SUMMARY_2K`: 326 tokens
- `REPOSITORY_ONLY`: 0 tokens

Although `SUMMARY_2K` and `STRUCTURED_NOTES_2K` had smaller initial handoff payloads than `KEISHO_2K`, their cumulative input tokens over the run approached `KEISHO_2K` (within 3–6%) because both required an additional full conversation turn.

5.2 Turn Efficiency and Model Calls

- **Joint-Fewest Calls**: Both `FULL_HISTORY` and `KEISHO_2K` completed the repair in **4 model calls** (3 tool calls). In both conditions, the successor examined the target files on Call 1, applied the patch on Call 2, executed validation tests on Call 3, and concluded on Call 4.
- **Verification Turn in Summaries**: Both `SUMMARY_2K` and `STRUCTURED_NOTES_2K` required **5 model calls** (4 tool calls). In both branches, Claude Opus 5.5 took an extra turn inspecting the codebase before applying edits, reflecting uncertainty regarding the unverified narrative claims.
- **Re-Exploration Overhead**: `REPOSITORY_ONLY` required **7 model calls** and **6 tool calls**, spending Calls 1–3 executing directory searches (`find` , `grep`) to locate the relevant methods.

5.3 Execution Time and Latency Observables

Observed wall-clock durations were:

- `FULL_HISTORY`: 32.7s
- `KEISHO_2K`: 34.5s
- `REPOSITORY_ONLY`: 53.8s

- `STRUCTURED_NOTES_2K`: 86.5s
- `SUMMARY_2K`: 86.8s

While `FULL_HISTORY` and `KEISHO_2K` finished in ~33–35 seconds, `SUMMARY_2K` and `STRUCTURED_NOTES_2K` required ~86–87 seconds. However, wall-clock latency should not be interpreted as a direct measurement of prompt processing time alone; it reflects a composite of network transmission, queue wait time at the provider API, and the number of round-trip model calls.

5.4 Financial Ledger and Multi-Attempt Accounting

Across all development and execution attempts, expenditures were tracked using a fail-closed spend guard enforcing a conservative non-cached price ceiling and a 5% provider fee multiplier.

Comprehensive Spending Summary Across All Attempts

Attempt	Model Calls	Status	Billed Spend (USD)	Non-Cached Accounted (USD)	Held / Unreconciled (USD)	Description
001	8	Aborted	$0.069326	$0.148785	$0.000000	Heuristic detector missed checkpoint
002	13	Aborted	$0.065966	$0.249584	$0.000000	Predecessor format bypassed regex
003	10	Aborted	$0.036516	$0.105225	$0.379008	Anthropic rejected `seed` with 404
004	14	Aborted	$0.344762	$0.414938	$0.000000	Tokenizer bound exceeded on Call 1
005	33	Completed	$1.534741	$1.590336	$0.000000	Successful five-way benchmark run
Total	**78**	—	**$2.051311**	**$2.508868**	**$0.379008**	Combined totals across 5 attempts

Breakdown of Completed Attempt (005)

- **Predecessor (`openai/gpt-6-sol`)**: 6 calls, 38,497 input tokens, 985 output tokens; Billed: **$0.031248**
- **Note Generation (`openai/gpt-6-luna`)**: 2 calls, 39,265 input tokens, 657 output tokens; Billed: **$0.005236**
- Summary Notes: 1 call, 19,546 in, 330 out ($0.002608)
- Structured Notes: 1 call, 19,719 in, 327 out ($0.002628)
- **Keisho Compiler**: Local deterministic compilation; Billed: **$0.000000**
- **Successor Branches (`anthropic/claude-opus-5.5`)**: 25 calls, 329,694 input tokens, 8,974 output tokens; Billed: **$1.498256**
- `FULL_HISTORY`: 4 calls, 138,443 in, 1,585 out ($0.585472)
- `SUMMARY_2K`: 5 calls, 44,126 in, 1,931 out ($0.215124)
- `REPOSITORY_ONLY`: 7 calls, 54,526 in, 2,315 out ($0.264404)

- `KEISHO_2K`: 4 calls, 47,170 in, 1,493 out ($0.218540)
- `STRUCTURED_NOTES_2K`: 5 calls, 45,429 in, 1,650 out ($0.214716)
- **Grand Total (Attempt 005)**: 33 calls, 407,456 input tokens, 10,616 output tokens; Billed: **$1.534741** (Accounted ceiling: $1.590336; Remaining headroom under $20.00 cap: $18.41).

---

6. Threats to Validity and Study Limitations

1. **Single-Task Scope ($N=1$)**: This study evaluates one specific SWE-bench Verified instance (`pydata__xarray-6721`). While it demonstrates feasibility and provides detailed empirical measurements of token volume, call counts, and failure reproduction, it does not support general claims of statistical superiority across broader benchmarks.
2. **Pre-Interception Fix Formulation**: When the predecessor was intercepted at Call 6, it had already formulated a candidate patch in its tool call arguments. Although the patch was never executed, this proposed solution was visible in the raw turns of `FULL_HISTORY` and described in `SUMMARY_2K` and `STRUCTURED_NOTES_2K`. Consequently, those successors were evaluating and applying a pre-formulated fix rather than diagnosing the issue from scratch.
3. **Frontier Successor Capability**: Claude Opus 5.5 was able to resolve the task even with zero context (`REPOSITORY_ONLY`), demonstrating that highly capable frontier models can independently re-solve this defect within 7 calls. Handoff representations primarily affected token efficiency and turn count rather than binary solvability.
4. **Absence of Successor Seed Pinning**: Due to Anthropic's provider rejection of the `seed` parameter via OpenRouter, successor runs operated without server-side seed locking. Despite `temperature: 0.0`, output variability across runs cannot be completely ruled out.

---

7. Conclusion

This study successfully executed a controlled five-way empirical comparison of context handoff representations during a mid-task takeover between asymmetric frontier models on SWE-bench Verified. By intercepting the predecessor prior to source-code execution and validating the unpatched state through both native harness grading and dedicated Docker test suite execution, the protocol established an empirically verified unresolved baseline.

All five successor conditions achieved task resolution (5/5 `resolved`). Across this single repair task, compiler-structured epistemic handoffs (`KEISHO_2K`) matched the turn efficiency (4 model calls) of uncompressed conversational history while reducing input token consumption by 65.9% (47,170 vs. 138,443 tokens). Free-form narrative summaries achieved similar token reductions (44,126 tokens) but required an additional exploratory model call and tool execution turn. These results illustrate the practical trade-offs between context compression, information fidelity, and execution overhead in multi-agent software engineering systems.

Reproducibility Appendix: Pinned Artifacts, Specifications, and Logs

Appendix A: Experiment Identifiers and Pinned Specifications

- **Governing Specification**: `swebench-handoff/2.7`
- **Pinned Specification Configuration**: `configs/swebench-comparison-v2.7.json` (SHA-256: `09f93905cfc9b0e12d5e23630d70bba713ebdc13ea0183bdafa7c24479cb25e2`)
- **Execution Plan**: `analysis/swebench-comparison-v2.7-plan.json` (SHA-256: `cc84d41d182e0578635e9851ec03c80e0c034220b3341c25fd79c657519bf7bc`)
- **Source Code Freeze**: `analysis/swebench-v2.7-final-source-freeze.json` (SHA-256: `088b71d497db3b8ec862086e115da7311749fe48a95f00e95ee7b43f0cfc134d`)
- **Completed Run Directory**: `runs/swebench-v2.7-five-way-once-005/`
- **Financial Authorization ID**: `bagi-20261027-v27-05` ($20.00 authorized ceiling)

Appendix B: Execution Records and Raw Trajectories

- **Run Manifest**: `runs/swebench-v2.7-five-way-once-005/experiment-manifest.json`
- **Primary Result**: `runs/swebench-v2.7-five-way-once-005/result.json`
- **Comprehensive Spending Ledger**: `runs/swebench-v2.7-five-way-once-005/spending.json`
- **Predecessor Grade Artifact**: `runs/swebench-v2.7-five-way-once-005/checkpoint-grade-private.json`
- **Task Docker Image Record**: `runs/swebench-v2.7-five-way-once-005/task-image.json`

Appendix C: Checkpoint Docker Verification Artifacts

- **Verification Execution Script**: `analysis/checkpoint_verification_command.sh`
- **Full Verification Pytest Log**: `analysis/checkpoint_verification_log.txt`
- **Structured Verification Result**: `analysis/checkpoint_verification_result.json`
- **Docker Image Digest**: `swebench/sweb.eval.x86_64.pydata_1776_xarray-6721@sha256:58c7f4ada7f3cbcfcde50e0b55157701100cf694faa87978c672e368c0531898`
- **Baseline Git Commit**: `cc183652bf6e1273e985e1c4b3cba79c896c1193`